TASK 2:

Deletion in an LLRB tree is a more complex process than insertion because the algorithm has to preserve both the black-height property and left-leaning red links. The fundamental concept behind LLRB deletion is to guarantee that deletion of a node never compromises the tree's invariant properties, particularly the property where all paths from root to leaf contain an equal number of black nodes. To accomplish this, LLRB deletion follows a top-down approach by proactively adjusting the tree as it travels downwards to reach the node to be deleted.

During its travel downwards, LLRB deletion checks if the current node or its child nodes form a 2-node, where a node has two black child nodes. This is a situation where deletion can compromise the invariant property of a balanced tree. Therefore, LLRB deletion transforms a 2-node into a 3-node or a 4-node by flipping colors and rotations. If LLRB deletion travels left and finds a left child and a left grandchild both of which are black, it performs a moveRedLeft operation. Similarly, if LLRB deletion travels right and finds a 2-node in its child nodes, it performs a moveRedRight operation.

Once this node is reached, the process is similar to that in standard binary search tree deletion. If this node has no right child, it is directly deleted. If it has two children, it is replaced by its in-order successor and then this successor is deleted using recursive calls. The advantage of this approach is that this node to be deleted is either red or has a red child due to the transformations carried out so far.

After deletion, a fix-up process is initiated while backtracking in the tree. This process involves using the same operations as in insertion to restore LLRB properties: left rotation to remove right-leaning red links, right rotation to remove consecutive left-leaning red links, and flipping red links when both children are red.

Overall, the deletion procedure maintains the left-leaning property by strictly preventing right-leaning red links through rotations and by carefully managing color distribution before and after removal. While effective, this process is considered more complex than insertion due to the need for proactive restructuring and multiple helper operations. This explanation is based on descriptions from Robert Sedgewick's original LLRB work and further clarifications from algorithm

Reference:

Algorithms, 4th Edition – Left-Leaning Red-Black Trees, Robert Sedgewick and Kevin Wayne

https://stackoverflow.com/questions/15455042/can-anyone-explain-the-deletion-of-left-lean-red-black-tree-clearly

https://cs.stackexchange.com/questions/19696
https://www.teachsolaisgames.com/articles/balanced_left_leaning.html

Task 3:

With Left-Leaning Red-Black (LLRB) trees, some interesting patterns emerge when they are compared with traditional RB trees. The rules are quite direct. If the red link leans to the right, rotate left and if two red links occur together, rotate right and if both children are red, flip colors. These rules are similar to those used with traditional 2-3 trees, where it is easier to visualize how keys are grouped together.

The process of deletion is more complex. Unlike insertion, where keys are simply inserted, deletion requires restructuring along the path to the key to be deleted. The process looks for 2-nodes, which are then converted to 3-nodes or 4-nodes using operations such as moveRedLeft or moveRedRight. After the key is found, it is then deleted, followed by rotations to maintain the left-leaning property. This is where the process becomes complex, with small errors having the potential to destroy the balance property .

Comparing LLRBs and traditional RB trees, we can observe many differences. For instance, LLRBs are more constrained data structures because they have left-leaning red links. This simplifies insertion because fewer cases have to be considered. For traditional RB trees, red links are allowed to lean in either direction. This makes insertion slightly more symmetric but also slightly longer in code. For deletion, traditional RB trees are more standardized and easier to implement correctly, compared to LLRB trees, which have a more complicated deletion process.

When comparing the data structures in terms of their efficiency, it has been observed that LLRB trees and traditional RB trees have the same time complexity of O(log n) in terms of insertion, deletion, and searching. LLRB trees do not provide a speed advantage but make insertion easier to understand and implement. Deletion in LLRB trees is still complicated, which offsets the advantage of insertion. As mentioned by Kohler in 2023, LLRB trees are interesting data structures that are useful in teaching and understanding the behavior of balanced trees but do not provide advantages in terms of complexity and speed.